UNIVERSIDAD DE GRANADA

/ UGR / decsai

Departamento de Ciencias de la Computación e Inteligencia Artificial

Inteligencia Artificial
Curso 2025-26

Práctica 1: Introducción a la IA Generatvia

Cuestionario sobre el Ejercicio 2:

Generación de Código para resolver el 8-puzzle con IA Generativa

Apellidos:	**Nombre:**
Grupo de prácticas:	Juan Reche Aguilar

Sigue el guión proporcionado para el Ejercicio 2 de la Práctica 1: Introducción a la IA Generativa y contesta las siguientes cuestiones.

¿Qué asistente has usado?

Gemini

Representación de estados y problema

Escribe en el recuadro inferior el texto que has usado para completar el prompt sobre **definición de un estado del 8-puzzle**

una tupla unidimensional de 9 números enteros, donde los elementos representan las casillas del tablero leídas fila a fila, de izquierda a derecha y de arriba a abajo.

Escribe en el recuadro inferior el texto que has usado para completar el prompt sobre el **método "representacion_problema"**

Entrada: no recibe parámetros externos aparte de 'self', ya que asume que la clase Problema se ha inicializado previamente con dos atributos (estado_inicial y estado_objetivo, ambas tuplas de 9 enteros). Restricciones: debe usar internamente la función de representación visual generada en el paso anterior. Qué debe hacer: formatear un texto que indique claramente cuál es el "Estado Inicial:" seguido de su matriz 3x3, y cuál es el "Estado Objetivo:" seguido de su matriz 3x3

Escribe en el recuadro inferior el texto que has usado para definir **la condición de estado objetivo**

la tupla de 9 enteros que representa el estado actual pasado como argumento es exactamente idéntica a la tupla almacenada en el atributo estado_objetivo de la clase Problema (es decir, estado_evaluado == self.estado_objetivo).

Describe brevemente, en el recuadro inferior, en qué ha consistido tu proceso de verificación para la parte de **representación de estados y problema**

El proceso de verificación consistió en crear un estado inicial con el uso del construtor y hacer un print del estado inicial y del final además de verificar que devolviera true o false debidamente la función para verificar si hemos acabado.

REPRESENTACIÓN DE OPERADORES

Escribe el texto, en el recuadro inferior, que has usado para completar el prompt **sobre representación de operadores.**

las letras 'U', 'D', 'L' y 'R'. Cada operador mueve la casilla vacía (representada por el 0) hacia Arriba (Up), Abajo (Down), Izquierda (Left) o Derecha (Right) respectivamente, intercambiando la posición del 0 con la ficha adyacente en esa dirección.

Escribe el texto, en el recuadro inferior, que has usado para definir las **precondiciones de cada operador**.

El operador 'U' es aplicable si el 0 NO está en la primera fila (índices 0, 1, 2 de la tupla). El operador 'D' es aplicable si el 0 NO está en la última fila (índices 6, 7, 8). El operador 'L' es aplicable si el 0 NO está en la primera columna (índices 0, 3, 6). El operador 'R' es aplicable si el 0 NO está en la última columna (índices 2, 5, 8).

Escribe el texto que has usado para definir la función **"Esaplicable<operador>"**.

Recibe como entrada el estado actual (una tupla de 9 enteros). Su salida debe ser un valor booleano (True o False). Lo que debe hacer la función es encontrar el índice donde está el 0 en la tupla y devolver el resultado de evaluar la precondición matemática descrita anteriormente para ese movimiento en concreto.

Escribe el texto, en el recuadro inferior, que has usado para definir **los efectos de cada operador.**

Genera una función independiente para el efecto de cada operador (ej. EfectoU, EfectoD, EfectoL, EfectoR). Cada una recibe como entrada el estado actual (tupla de 9 enteros). Debe calcular el nuevo estado intercambiando el valor 0 con la ficha de destino (restando 3 a su índice para U, sumando 3 para D, restando 1 para L y sumando 1 para R). Su salida será una nueva tupla de 9 enteros representando el estado resultante.

Escribe el texto que has usado para **describir la entrada y salida** de la función de transición "AplicaOperador".

Recibe como entrada el nombre del operador (una cadena de texto 'U', 'D', 'L' o 'R') y el estado actual (una tupla de 9 enteros). Primero debe llamar a la función 'EsAplicable' correspondiente para comprobar si el movimiento es válido. Si lo es, calcula el nuevo estado; si no lo es, devuelve None. La salida será el nuevo estado (tupla de 9 enteros) o None.

Escribe el texto que has usado **para definir la función** de transición "AplicaOperador".

Evalúe qué operador de texto ha recibido como parámetro y, utilizando condicionales (if/elif) o un diccionario, invoque a la función específica 'Efecto<operador>' correspondiente (EfectoU, EfectoD, EfectoL o EfectoR) que hemos definido en el paso anterior, devolviendo la tupla que estas funciones calculen.

Describe brevemente, en el recuadro inferior, en qué ha consistido tu proceso de verificación para la parte de **representación de estados y problema**

Primero, partiendo de un estado con el hueco en el centro, apliqué los cuatro operadores posibles ('U', 'D', 'L', 'R') comprobando por pantalla que las fichas se intercambiaban correctamente. Finalmente, probé a forzar un movimiento ilegal (intentar subir el hueco cuando ya está en la fila superior) para asegurarme de que la función evaluaba bien las precondiciones y devolvía 'None' en lugar de dar error.

GENERACIÓN DEL ESPACIO DE ESTADOS

Escribe el texto, en el recuadro inferior, que has usado para completar el prompt **sobre generación del espacio de estados**

Esta función recibe como entrada el estado actual (una tupla de 9 enteros). Su salida debe ser una lista de tuplas, donde cada elemento de la lista sea un par (nombre_del_operador, estado_resultante). Para calcular esto, la función debe crear una lista vacía de sucesores e iterar sobre los cuatro operadores posibles ('U', 'D', 'L', 'R'). En cada iteración, llamará a la función AplicaOperador (importada de acciones.py) pasándole el operador y el estado actual. Si la función devuelve un nuevo estado válido (es decir, distinto de None), añadirá el par (operador, nuevo_estado) a la lista de sucesores. Al terminar, la función devolverá la lista completa.

Describe brevemente, en el recuadro inferior, en qué ha consistido tu proceso de verificación para la parte de **generación del espacio de estados.**

Definí un estado de prueba situando el hueco (el 0) en la esquina superior izquierda del tablero (estado '012345678'). Al ejecutar la función EstadosSucesores, comprobé por pantalla que el programa solo generaba y devolvía los dos únicos movimientos válidos posibles desde esa esquina (Abajo 'D' y Derecha 'R'), descartando correctamente los demás y mostrando el tablero resultante para cada acción.

SOLUCIÓN DEL PROBLEMA

Escribe el texto que has usado para especificar la entrada, y salida de la función **"Soluciona8puzzle"**.

Que reciba como entrada una instancia de la clase Problema (que contiene el estado inicial y el estado objetivo). Su salida debe ser una lista de cadenas de texto (por ejemplo, ['U', 'L', 'D', 'R']) que represente la secuencia de operadores para llegar a la solución, o None si no hay solución. Para implementarlo de forma modular, el algoritmo debe usar una estructura de datos tipo Cola (FIFO) para la frontera de exploración (donde cada elemento guarde el estado actual y la lista de operadores aplicados hasta llegar a él) y un conjunto (set) para registrar los estados ya visitados y evitar ciclos.

Describe brevemente, en el recuadro inferior, en qué ha consistido tu proceso de verificación para la parte de **solución del problema.**

Como ya era la ultima parte el proceso de verificación a consistido en probar el programa "final" con una entrada como la de los ejemplos en el apartado de experimentación y ver con la ayuda de las funciones de los apartados anteriores si la solución se veía correcta en la terminal.

EXPERIMENTACIÓN

Ejecuta varias veces la función de solución del 8 puzzle y rellena los datos de las siguientes tablas para los siguientes problemas

Estado inicial	Estado objetivo	Número de pasos
123485706	123456780	3
283175460	123456780	10
213456780	123456780	
357201468	123456780	20
123456870	123456780	20

Estado inicial	Estado objetivo	Solución
123485706	123456780	['U', 'R', 'D']
283175460	123456780	['L', 'U', 'U', 'L', 'D', 'D', 'R', 'U', 'R', 'D']
213456780	123456780	No existe
357201468	123456780	['R', 'U', 'L', 'L', 'D', 'R', 'R', 'D', 'L', 'U', 'R', 'U', 'L', 'L', 'D', 'D', 'R', 'U', 'R', 'D']
123456870	123456780	['R', 'U', 'L', 'L', 'D', 'R', 'R', 'D', 'L', 'U', 'R', 'U', 'L', 'L', 'D', 'D', 'R', 'U', 'R', 'D']

Explica brevemente qué conclusiones extraes de la ejecución, especialmente sobre si se encuentra solución a todos los problemas y por qué, si piensas que la solución que obtiene es la óptima (es decir, con el menor numero de pasos posible) y por qué.

En primer lugar, no se encuentra solución a todos los problemas. Matemáticamente, el 8-puzzle se divide en dos espacios de estados separados; por tanto, existen tableros iniciales que son insolubles (imposibles de resolver) debido a la paridad de sus inversiones. Si el algoritmo recibe uno de estos, explorará todas las combinaciones posibles sin éxito. En segundo lugar, cuando el algoritmo sí encuentra una solución, ésta es siempre la óptima (con el menor número de pasos posible). Esto se debe a la naturaleza del algoritmo empleado: la Búsqueda en Anchura (BFS) explora el árbol de estados nivel por nivel, lo que garantiza que el primer camino que llegue a la meta será estrictamente el más corto.